Coding Standards and Quality Requirements for Development

This page is linked to Development and Delivery Process

This page aims to give all our developers a common set of best practices and guidelines for development. This is both to ensure quality in our deliveries, ensure secure code, as well as ensuring that the code you write is future-proof.

no matter if the delivery is a new module in standard, the core 360° platform, or a customization for customers.

Target audience: All developers in R&D/Core and Technical Consulting. These guidelines must be followed by ALL developers in both R&D/Core and Technical Consulting. If you feel there's something missing, or the guidelines prevent you from doing your work in an efficient way, please raise this issue in the Software Architecture Skill Group channel. We'll discuss improvements to the guidelines, and guide you on the issues at hand. **It does not matter if you're working on custom code for a customer, or working on the standard product! This applies to EVERYONE!**

We have a recording available from a walkthrough of this page. Find this **HERE (https://tietocorporation.sharepoint.com/:v:/r/sites/TechnologyDevelopmentSI/Shared%20Documents/General/A%20briefing%20of%20our%20Coding%20Standards%20_%20Quality%20Requirements%20(Mandatory%20for%20developers).mp4?csf=1&web=1&e=cUPGn1)**

Contents

General coding standards

Ensure to follow established procedure when adding project references in core-bl

For some time ago we've switched all references to dlls in BinaryOutput to project references. This means that there are some important points that developers must account for when adding project references to the projects, especially references to projects residing in other solutions:

- **DO NOT** add references as assembly references to BinaryOutput anymore
- **DO** use project references. If you refer to a project in a different solution than the one you are working in, be sure to add that project to your current solution, to the "ReferencedProjects" solution folder.
- **DO NOT** manually update the core-bl.sln, core-bl-net48.sln or the core-bl-net5.0.sln solutions. Use the Update-AllSolutionFiles.ps1 script for this.

Important note: if you're building a solution locally and get errors like "Metadata for [PROJECT_NAME] not found", please ensure that the project reference for it is properly added. If the project that is not found is a part of another solution, ensure that it's added to "ReferencedProjects" solution folder.

Ensure proper filehandling

If your code needs to handle files other than 360 files, ensure to follow the "360 file handling" guidelines

Storing sensitive information and credentials

Sometimes, we need to store credentials for third-party services. These should be stored securely (encrypted) in pv_propertyvalues. **TODO - Geir Are**

Naming and formatting standards

C# naming standards

Properties (both private and public)
Use PascalCase (also called Upper Camel Case) when naming properties in C#. This means each word that is part of the name should start with a capital letter. Example: MyProperty.

Class member variables (fields)
Use camelCase, with a leading underscore. For instance:

```
int _myPrivateVariable;
```

. All fields should be private unless there's a solid reason. If you need a public variable, use properties.

Local variables
Use camelCase, without a leading underscore. For instance:

```
int myLocalVariable = 5;
```

.

Functions/methods (both private and public)
Use Pascal case, same as for properties.

Tabs or spaces

Source control and tools

Azure DevOps is our tool for builds, source control, testing, work item planning and release management.

Git is our standard source control system, hosted in Azure DevOps. Only legacy components, or code that are not yet migrated to git, are still in TFSVC. Everything NEW must use Git. Everything that is being updated regularly must be in Git.

Ensure all code you write fulfill the principles of privacy by design and GDPR

See: https://en.wikipedia.org/wiki/Privacy_by_design

It's crucial that you know what this actually means - if you do not, please speak up on Teams.

Do proper checks of input arguments to functions

Never assume that input arguments to functions are valid. Expect that they can be of ANY value, and handle them properly. This includes, but are not limited to, null checks.

Data types

Please keep all values as their "native" data type as much as possible. Avoid converting dates, numbers, booleans etc. to strings just because it seems easier to handle that way.

Whenever you really *do* need to convert to string; do not rely on the default "automatic" string conversion. Values converted to or from string representation need to use the correct formatting. Some use-cases must use culture-invariant formatting, and some use-cases must use culture-specific formatting. Know the difference. Generally; for display in the UI, you must be culture-specific. For any technical usage, such as XML serialization, config files, passing parameters to other APIs, you must be culture-invariant.

Take special care with date/time values. Here we must generally always specify a specific date format for each use-case. **Never call `ToString()` without arguments**! And again, we need to choose correctly between culture-invariant or culture-specific format depending on the scenario.

In user interfaces; use the specific input controls for the data type you are handling; don't just use the default textbox for everything. Both BIF and HTML have specific inputs for dates, numbers, URLs, email, phone numbers, etc.

Avoid storing large amounts of data in memory at once

Whenever you need to process a stream that may contain a large amount of data (1MB or more), **do not** copy it to a MemoryStream, or read it into a byte array, or read it into a string, or any other datatype that requires all the data to reside in memory at once. Instead, consider the following options outlined in the following page: How to process large streams without loading them into memory.

Storing and managing configuration

Avoid using configuration stored in configuration files (web.config, workunits, etc.) as much as possible. Instead, all configuration should be stored in the database. The reason is that any changes to configuration files adds complexity to the startup procedure for 360° containers. Additionally, configuration stored in the database can later be made available through the web UI 360° API, if applicable.

Please refer to this Move sub- and module-setup configurations to the database for a how-to and a practical example.

Prefer non-static classes instead of static classes

Static classes are hard to mock. As such, unless it's a class that is pure logic and thus never needs to be mocked, please prefer to create a regular non-static class. Optionally, you can include a static accessor to implement a singleton-like pattern. This achieves:

1. You can inherit and mock the class so that it can be used as a dependency in a unit test.
2. With a static accessor, you can still use the class just like a static class when needed.

So instead of this:

```
public static class MyHelper
{
    public static string GetDataFromBL()
    {
        var response = ...; // do a HTTP request here
        return response;
    }
}
```

Do this:

```
public class MyHelper
{
    private static object _lock = new object();
    private static MyHelper _instance;
    public static MyHelper Instance
    {
        get
        {
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                        _instance = new MyHelper();
                }
            }
            return _instance;
        }
    }

    public virtual string GetDataFromBL()
    {
        var response = ...; // do a HTTP request here
        return response;
    }
}
```

There's a little bit more code, but it can be more or less copy-pasted to your class.

Now to use it, you can follow the pattern below. This both allows you to use the class without worrying about dependency injection when using it in "production", while allowing mocking of the helper class using dependency injection in unit tests:

```
public class MyOtherClass
{
    MyHelper _myHelper;

    public MyOtherClass()
    {
        _myHelper = MyHelper.Instance;
    }

    public MyOtherClass(MyHelper myHelper)
    {
        _myHelper = myHelper;
    }

    public string MyMethod()
    {
        return _myHelper.GetDataFromBL();
    }
}
```

Refactoring/larger rewrites/breaking changes

When doing refactoring of existing code, larger rewrites, or introduce breaking changes, please follow this guide: A guide to refactoring and breaking changes.

Create new functionality as a microservice, if possible

Microservices is quite new - but some types of functionality can sometimes be implemented much easier as a microservice instead of extending our existing BL. In these cases, please consider creating a microservice for that functionality. Please see: How to create a microservice with everything included

Guidelines for containerized applications hosted on Kubernetes

All applications that are going to be deployed as containers to Kubernetes must follow established guidelines to ensure that all the deployments are performed in a controlled, sustainable, secure and available manner. Guidelines are defined here and are continuously updated: Development guidelines for containers and Kubernetes deployments

Always verify length of the file path

We're often using temporary files and storing them in a temporary directory on the 360 application server. This needs to be done with care, especially when it comes to our cloud solution where tenant id is a 36-char guid and a part of the application pool user name and such the path name. Such temporary paths can get pretty long which will result in "Path too long" exceptions when it's larger than the Windows operating system limit of 260 characters.

If you're using a combination of `LocalTempPath()` and `Guid.NewGuid()` and adding filename on top of it without any path length validations, there is a pretty high chance that at some point the customer will experience the error mentioned above. Therefore, it's extremely important to perform a validation of the file path length as part of path concatenation in order to avoid errors like this in the product. Always do a total file path + file name check and if it's larger than 260 chars, concatenate and shorten the filename.

Example of a path in an Online environment that is returned when using `LocalTempPath()` (76 chars)

C:\Users\SI.WS.Core_6c4e77aa-ee64-41cb-b4b6-56678e01603d\AppData\Local\Temp

And as a final thought: Do you really *need* to store the files on the 360 app server or can you stream the directly? Check PR 30123 (https://tieto-si.visualstudio.com/360/_git/core-bl/pullrequest/30123) to see such a rewrite that fixed a "Path too long" error.

Logging

Logging is a powerful tool for getting an overview of application's behaviour in addition to making debugging a much more efficient process. Logging provides vital data on the real-world use of your application. When things stop working, it's the data in the logs that both the development and operations teams use to troubleshoot the issue and quickly fix the problem. Unfortunately, through the years different tools and places have been used for logging, in addition to that quite a lot of unnecessary information was logged, which makes it a challenge to filter false positives as well as introduce "smart" monitoring for the Operations team. Now, that we're moving towards containers, Kubernetes and microservices we must build a strong observability platform for our application. In order to succeed we must follow best practices when logging information about our application so that we have one place to retrieve the logs from, as well as ensuring that the logs contain the right and relevant information when we need it. Following guidelines will hopefully make it easier for you to evaluate what kind of information you need to log in your use case and when you should log what.

- **Don't log personal sensitive information**: ensure that you are not logging potentially sensitive information that can violate GDPR. Example of such information may be social security number. You can read more about what personal data is here: What is personal data? (https://ec.europa.eu/info/law/law-topic/data-protection/reform/what-personal-data_en)
- **Avoid extensive logging**, ensure that you're logging only that information that is necessary and relevant for debugging purposes or evaluating how application performs.
- **Logging in BL**: Don't log directly to Event Log, always use SI.Util.Diagnostics library. Event Log is a Windows exclusive tool and will cease to exist when 360 Business Logic is ported to .NET Core and hosted on Linux. Even now, when we're running 360 application in a Docker container, retrieving logs from Event Log is a time-consuming and complicated operation which requires many manual steps. So, never use System.Diagnostics.EventLog library directly.

Don't do this:

```
EventLog.WriteEntry("LogMyWarning", "Writing warning to event log.", EventLogEntryType.Error, myEventID, myCategory,
myLogData);
```

Instead, do this:

```
LogWriter.WriteLine("LogMyWarning", "event", "Writing warning event to log: " + myLogData, DiagnosticSeverity.Warning);
```

- **Logging to ilog_integrationlog**: use logging to ilog_integrationlog only when you need to log information related to integrations. Use ilog_integrationlog when you expect a specific request coming from an integration to fail, for example, when request is missing necessary information or contains incorrect information. Another example when logging to ilog_integrationlog can be used is when you need the user to perform a specific action in order to successfully complete the request - for example, add missing configuration.
- **Logging in microservices:** always use ILogger from Microsoft.Extensions.Logging library for logging in microservices. Observability framework in 360 will be implemented based on OpenTelemetry which will automatically collect all the logging implemented with ILogger. You can read more about how to use ILogger in official Microsoft documentation: Logging in .NET Core and ASP.NET Core (https://docs.microsoft.com/en-us/aspnet/core/fundamentals/logging/?view=aspnetcore-5.0)
- **Ensure that you understand what log levels exist and when each of these levels is applicable.** A very brief and understandable overview is provided in official Microsoft documentation: Log levels (https://docs.microsoft.com/en-us/aspnet/core/fundamentals/logging/?view=aspnetcore-5.0#log-level)
-

Porting existing code into standard

Sometimes, there are existing solutions that have been delivered as a "custom" delivery package that are being moved into standard as a module. These existing solutions could be reusable concepts, custom integrations, etc.

In the standard product we aim to keep a high quality standard for any new code. As such, all the coding standards and quality requirements listed on this page is applicable when moving the code into standard. This may require some rewrites in the code that is ported. Examples are:

- If there's no unit tests, the code must be made testable, and high quality tests must be written. This both helps to prevent regression when other parts of the product change, ensure correctness and helps understanding of the code that is being committed to our standard repositories.
- Feature toggles must be in place.
- Custom meta must be moved to standard, where recnos are below 100000.
- All nuget packages must use the same version as the rest of the standard product. Typically, you will find the version listed in Packages.props in the root folder of the repository. You do NOT specify a version in the project file itself.
- Whenever there are custom BIF, the preferable way to port this is to rewrite the views into React-based views. If this is not possible (e.g. there are additions to existing BIF views, like wizards), then the code must be ported into standard BIF.

This is NOT a complete list. **It's expected from the developer that all the coding standards and quality requirements listed on this page is followed**. From experience, porting code into standard takes time, usually more than you initially expect. It's **NOT** a matter of just copy-pasting the code into the standard repositories.

Git Branching Strategy

We have defined our branching strategy in the wiki page Branching Strategy for Standard Development.

.NET Coding Best Practices

Think Cross-Platform

Although we currently do not run 360° on anything other than Windows Server, we have ambitions to running it on Linux in the near future. For both **new and old projects**, please follow the .NET Core Porting Guide to ensure that your project is cross-platform and .NET Core compatible.

In short:

- Target both .NET Framework 4.8 AND .NET 6 for your class libraries if you are developing for our current BL or frontend. If not, ONLY target .NET 6.
- For new executables, tools or unit test projects; use .NET 6.
- Always use the new simplified csproj format for the project file. See the .NET Core Porting Guide.

    - The easiest way, when you create a new project, is to select "Class Library (.NET Core)". When the project is created, open it in an editor and change the value of <TargetFrameworks> to `net48;net6.0`
- **Do not** use Windows only features like e.g. registry, WCF. For settings, use the database (https://wiki.software-innovation.com/wiki/Move_sub-_and_module-setup_configurations_to_the_database). For SOAP, wrap them using SoapWrapper (https://wiki.software-innovation.com/wiki/Design_Standards_%26_Guidelines_for_360%C2%B0_Extensions_and_Integrations#SOAP_Integrations_and_Services).
- Constructing file paths:
    - **Instead of:** `string myPath = rootPath + "some-file\\over-here.txt";`
    - **Use this:** `string myPath = Path.Combine(rootPath, "some-file", "over-here.txt");`
    - If for **any** reason you can't use Path.Combine, use forward slashes: `string myPath = rootPath + "some-file/over-here.txt";`. **But always prefer Path.Combine whenever you can!**
- Treat all file paths as **case sensitive**. For instance, if you have a file "foo/Bar.sql" in a delivery package, the package.xml entry that refers to this file must match "foo/Bar.sql" exactly. NOT "foo/bar.sql" for instance. The Linux filesystem is case sensitive, and since we are moving there, everything we make should take this into account.

    Additionally - use lower case in the filename for "package.xml" itself in your delivery packages.
- Avoid terminating streams. When reading files from the BL for instance, the streams returned are non-seekable and not allocated in memory. Not terminating streams is extremely important when working with large files of course, but the pattern should be valid in most scenarios.
- If you need some code to be compiled only for .NET Framework, or only for .NET Core, use the following construct:

```
#if NETFRAMEWORK
// .NET Framework-specific code here
#else
// .NET Core-specific code here
#endif
```

- If you need to include a NuGet package or reference only for .NET Framework, or only for .NET Core, use the following pattern in your project file:

```
<ItemGroup Condition="$(TargetFramework.StartsWith('net4'))">
  <Reference Include="System.ServiceModel" />
  <PackageReference Include="Swashbuckle" />
</ItemGroup>
<ItemGroup Condition="!$(TargetFramework.StartsWith('net4'))">
  <PackageReference Include="Swashbuckle.AspNetCore" />
  <PackageReference Include="SoapCore" />
</ItemGroup>
```

Always use culture when serializing/deserializing numbers and dates

When serializing (e.g. `ToString()`) or deserializing (parsing, e.g. `Parse` or `TryParse`) numbers and dates; always specify culture. For dates, also specify the format.

Date formats can vary widely between different cultures. The order in which day, month and year appears is different, and the separators are completely different too. For numbers, the decimal separator and negativity symbol can vary.

When you are rendering a user interface, specify the local culture. `CultureInfo.CurrentCulture` for dates and numbers, and `CultureInfo.CurrentUICulture` for localized strings. However, local cultures are subject to change by user customization or by updates to the .NET Framework or the operating system, so they are not good for consistent and reproducible results.

If you serialize or deserialize dates/numbers for JSON, XML, SQL or similar, always use `CultureInfo.InvariantCulture`. This is a special culture that is stable over time and across installed cultures and cannot be customized by users, so it will always give the same result.

If culture is not specified, .NET will implicitly use local culture. This will give incorrect results for everything except UI rendering. So, **for number types; never call ToString without the culture argument. For dates, never call ToString without format *and* culture arguments.** Likewise, **never**

let numbers and dates get implicitly converted to string when doing string concatenation, because format and culture info will be missing.

```
string str = "";

/**
 * Numbers (int, float, double...)
 */
int myNumber = -42;

str += "" + myNumber; // INCORRECT: Implicit conversion to string. Missing culture
str += Convert.ToString(myNumber); // INCORRECT: Missing culture
str += myNumber.ToString(); // INCORRECT: Missing culture
int parsedInteger = int.Parse(str); // INCORRECT: Missing culture

str += myNumber.ToString(CultureInfo.InvariantCulture); // CORRECT
str += Convert.ToString(myNumber, CultureInfo.InvariantCulture); // CORRECT
parsedInteger = int.Parse(str, CultureInfo.InvariantCulture); // CORRECT

/**
 * Dates
 */
DateTime myDate = new DateTime(2022, 3, 28, 10, 32, 0);

str += "" + myDate; // INCORRECT: Implicit conversion to string. Missing format and culture
str += Convert.ToString(myDate); // INCORRECT: Missing format and culture
str += myDate.ToString(); // INCORRECT: Missing format and culture
DateTime parsedDate = DateTime.Parse(str); // INCORRECT: Missing format and culture

str += Convert.ToString(myDate, CultureInfo.InvariantCulture); // INCORRECT: Missing format
str += myDate.ToString(CultureInfo.InvariantCulture); // INCORRECT: Missing format
str += myDate.ToString("s"); // INCORRECT: Missing culture
parsedDate = DateTime.Parse(str, CultureInfo.InvariantCulture); // INCORRECT: Missing format

str += myDate.ToString("s", CultureInfo.InvariantCulture); // CORRECT
parsedDate = DateTime.ParseExact(str, "s", CultureInfo.InvariantCulture); // CORRECT
```

Web Development

For quality requirements and coding standards for web code (javascript, HTML, ...) please see Frontend guidelines for modern web applications and Guidelines for accessibility. And don't forget; always Escape string values correctly!

BIF development

DO NOT create new functionality using BIF. Exceptions will only be made on case-by-case basis.

See: BIF development best practices. Since BIF is a type of web development, you must also be familiar with the guidelines in the **Web development** section above.

React development (FIB)

See FIB development best practices (https://fib.360online.com/?path=/docs/general-information-fib-development-best-practices--docs) and BL APIs for React (https://fib.360online.com/?path=/docs/general-information-client-callable-bl-apis--docs). Since React is a type of web development, you must also be familiar with the guidelines in the **Web development** section above.

Database changes

See Best practices for database scripts

Email and email templates

Security Requirements

Escape variables used for building SQL, URIs, XML, HTML, JSON etc...

Whenever you need to build a URI, SQL query, XML, HTML, JSON, etc. from variables that comes from settings, user input or any other external source, remember to Escape string values correctly! If you are in doubt, Escape string values correctly!

Always use HTTPS for calls to external systems

When making service calls to other servers; whether third party or our own; using an encrypted connection (https) is mandatory.

Likewise; when referring web resources from other parties or other servers; (such as scripts or icons from content delivery networks, map or chart data from a web-based API, iframes loading web pages from other servers) https must be used as well.

Whenever the URL to such a service endpoint or web resource is configurable (in web.config, the settings table, etc.) your code should verify that a https-based URL is used.

Should there be a need for unencrypted http in certain scenarios (development or test; for instance) you can add a setting to permit this. The name of the setting should indicate that it is insecure to use (for instance; "Allow insecure connections") and should be specific to one service endpoint URL. That is, if your code uses multiple URL settings; you need separate "allow insecure connections" settings for each of them.

The "allow insecure connections" settings must be default off; and should not be automatically enabled by the setup. The idea is that support for unencrypted http is only switched on when the deployer or customer makes an active decision to do so.

This is all especially important for connections that include credentials, or other personal or confidential data. But follow the same procedure for all http-based communication.

Unencrypted http should be considered deprecated. A last resort, or for development and test only.

Escaping

Parameters inserted into SQL queries; XML; HTML; JSON; URLs etc. must always be escaped correctly.

Avoid using string concatenation if possible. For instance; use a proper XML API to construct XMLs such as meta queries, instead of just concatenating strings.

Secure coding practices

Follow the guidelines in the article Secure coding practices

Build validations related to security controls

Follow the guidelines in the article How to resolve build validation errors related to security scans

Avoid adding third-party dependencies (or follow the process when adding new third-party dependencies)

We must **not** add any third-party dependencies to our source code unless it's absolutely necessary and unavoidable. Third-party dependencies introduce potential security risk since we don't have control over the code of the dependency and therefore we can't be 100% sure that it is secure and has high quality.

As long as it is possible to implement the needed functionality ourselves or utilize standard libraries provided by the framework itself or use libraries provided by trusted organizations like Microsoft - we must do that.

If none of the mentioned actions are possible and you must use a third-party library, **you're responsible to follow the process and ensure that all the required actions are done before publishing the PR!**

Before adding new third-party library to your code:

1. Reach out to the Architect team **as early as possible in the development cycle**! Architect team will evaluate and approve the new library or suggest a different implementation. If new third-party dependency is approved, you must update following documentation in Wiki: Third party software used in 360°. **If you don't bring this up before creating the PR where an external library is included, you must expect change requests and potential rejection of the PR in case the library is not secure or trustworthy enough.**
2. Check if third-party assemblies are signed or not. **If new third-party dependencies don't have code signing, reach out to the Architect team!** Missing code signing is a potential security risk since there is no way we can verify that the code is coming from valid and secure originator. In addition, we will not be able to verify if code/package has been tampered in-between. We have implemented a validation for the amount of unsigned assemblies in the build pipelines and the build will fail in case more unsigned assemblies are included in the source code. If unsigned assemblies are approved, you must whitelist the assembly by adding it's name to unsigned-dll-whitelist file in common-git-version-increment repository: https://tieto-si.visualstudio.com/360/_git/common-git-version-increment?path=/ReportFiles/unsigned-dll-whitelist.txt.

In order to start the process of evaluation and approval for the new library, we recommend all the developers to inform your closest Tech Lead about the new library and what it is supposed to be used for so that this information can be presented and evaluated by the Tech Lead in the Architect & Tech Lead meeting.

Product Quality Requirements

Testing

Developers should follow the Test Driven Development process (TDD). All new or modified code must have unit test coverage. Additionally, a selection of integration tests that covers the most important areas of the public interface to the code must be written.

For important details on writing tests and the test processes, please visit this wiki page: Core-testing process

For information about TDD, there's tons of resources online, but a good starting point is the Wikipedia article about TDD. (https://en.wikipedia.org/wiki/Test-driven_development) and internal Wiki article Test-driven development

Testing large changes

It is important to test large rewrites before merging to master, to keep master branch stable. To test large rewrite developers required automated way of getting complete install kit /container image with dev branch binary, create test environment and run UI & Integration test in automated way. To know the process of testing large rewrite visit wiki

Testing changes to 360 clients

If you're adding changes to core-clients repository or to SI.Biz.Core.ClientAccess project, it's important that you perform necessary testing of 360 clients to ensure that latest clients, as well as clients with 2 previous versions, are still working as expected. It means that if changes are added to version 5.10, then the client on 5.10 as well as 5.9 and 5.8 must be tested! Please remember to uninstall current client in order to install the clients on different version.

Code reviews and pull requests

All code that is merged to the master branch in any repository that contains production code in Git MUST be reviewed by someone who knows the code. This can be the team leader, or any other expert in the team. Code is typically reviewed through pull requests.

When setting up a new git repository; always set a branch policy on the master branch that enforces rules for pull requests. These rules should include mandatory code review, the build should pass, tests should pass, etc.

Please read Pull request guidelines and Guidelines for bypassing when completing pull requests.

Pull request and Git repo owners

To optimize Pull Request review period Owner identified for the core Repos. If your PR is unattended or need attention from reviewer, please contact Repo owner. Owner will make sure it is reviewed.

Repo Name	Owner
core-setuptools	Arne Smebye (mailto:arne.smebye@tietoevry.com)
core-bl	Thor Rudi (mailto:thor.rudi.com)
core-db	Rune Granerud (mailto:Rune.Granerud@tietoevry.com)
core-360Frontend	Tor Lye (mailto:Tor.lye@tietoevry.com)
core-bifeditor	Tor Lye (mailto:Tor.lye@tietoevry.com)
core-bifstore	Tor.Lye (mailto:Tor.Lye@tietoevry.com)
core-clients	Steinar.solhaug (mailto:steinar.solhaug@tietoevry.com)
core-360-react-views	Tor Lye (mailto:Tor.lye@tietoevry.com)
core-subsetup	Arne Smebye (mailto:arne.smebye@tietoevry.com)
core-ui-test	Mohan.kumar (mailto:mohan.kumar@tietoevry.com)
core-help	Per.borgersen(PC) (mailto:per.borgersen@tietoevry.com)
core	Thor Rudi (mailto:thor.rudi@tietoevry.com)
core-utils	Thor Rudi (mailto:Thor.Rudi@tietoevry.com)
core-bl-web	Tor Lye (mailto:tor.lye@tietoevry.com)

Performance Requirements

All code must be written in performance in mind. That's a challenge, especially where customers has a LOT of data, or many active users. When writing code, please follow the wiki page Coding for Performance.

Setup/delivery package requirements

- For the standard (main) 360° setup, the majority of the setup logic must be part of one of the core PowerShell setup scripts. This allows the setup logic to be run also on the 360° Docker container image.
- If you're adding new subsetup, new wizard variables or changing existing wizard variables **do ensure that setup configuration is properly handled for cloud installations also - both VM-based and container-based**! Let's say you have two setup tasks: one that copies files to a folder and one that deletes the contents of the same folder. You're using a switch that will run one of these tasks depending on a value of a specific wizard variable. In this case, if you don't verify that the variable is configurable and added to the standard modules definition in management portal, the container-based setup will just ignore thew switch part and first run the copy task to copy the files to the folder followed by a cleanup task to remove the contents of the same

folder. You can add a new wizard variable to an existing subsetup, change an existing wizard variable or add a new subsetup to this file: https://tieto-si.visualstudio.com/360/_git/core-cloud-automation?path=%2FSI.CloudOps.Dashboard%2FMigrations%2FStandardModules.json&version=GBmaster&_a=contents. You can also reach out to Cloud Automation team in case you need help for the same.
- Avoid using the Web Config Merger for doing web.config modifications. Instead, use the PowerShell library that we have for doing XML operations on configuration files, which replicates most of the functionality, while adding a bit more.
- Do **NOT** use webscripts in the package XMLs. Support for these will be removed in the future, and will certainly not work on the 360° Docker image.

See Deliveries to Cloud customers and the review process behind and Developing for Standard 360° for more details.

See Move sub- and module-setup configurations to the database for how to store and handle configurations.

Integrations and delivery package coding

We have a separate page outlining the standards and guidelines for delivery packages and integrations. Please visit that if you are writing a delivery package, and/or integrations against external systems.

Process Engine & processes / work units

See Process Engine

Code Review Requirements

Expectations from senior developers and technical leads

All tech leads and senior developers with sufficient experience are expected to review pull requests on a daily basis, even pull requests that doesn't belong to YOUR team.

Code review checklist

The following items must be taken into consideration when doing code reviews:

- Feature toggles MUST be in place for new code. For more info on feature toggles, see Feature Toggles.
- Automatic tests MUST be in place. The reviewer is responsible for ensuring that **sufficient** automatic tests are in place.
- Check for correctness, and check that best practices are followed (refer to this wiki article that you are reading now).
  - For example; web frontend code (HTML, JavaScript, ASP.NET...) must follow the frontend best practice. BIF files must follow the BIF best practice, and so on.
  - A BIF PR that includes in-line scripts or HTML templates must follow best practice for **both** BIF and frontend.
- Check that unintended changes or files are included in the pull request. For instance, local paths in various files, build output (DLLs, etc.).
- Check that credentials and other sensitive information is not committed.
- Check that logging is performed according to our best practices: Design Standards & Guidelines for 360° Extensions and Integrations#Logging.
- Check that the code follows the way of working defined in Developing standard modules for 360°.
- For pull requests that are targeting hotfix/release branches, code reviewers must verify that the change was cherry-picked from master. It is important to ensure that the change is present on master first, and the changes are identical (where possible) and correct on the relevant branches.
- If new third-party services or libraries are added or deprecated as part of the PR, this third-party service or library must be added or updated in the list of third party libraries and services on the wiki .If third-party assemblies are unsigned, they must be whitelisted in unsigned-dll-whitelist file in common-git-version-increment repository: https://tieto-si.visualstudio.com/360/_git/common-git-version-increment?

path=/ReportFiles/unsigned-dll-whitelist.txt. **First and foremost, new third-party libraries must be evaluated and approved by the architects and tech leads - if this hasn't been done, the PR must be put on hold or rejected until the library is evaluated!**
- If MemoryStreams or byte arrays are used, or if reading a stream into memory, ensure that the amount of data involved is small. E.g. if loading an XML with embedded file data, then this must not be loaded into memory at once. See How to process large streams without loading them into memory.
- If new project reference is added to a project in core-bl, ensure that it's added according to best practices. If the project reference is for a project residing in another solution than the project where it is added to as a project reference - ensure that it's also added to "ReferencedProjects" folder!
- If you're reviewing PRs in **core-clients** repo or PRs that include changes in SI.Biz.Core.ClientAccess project in core-bl, ensure that these changes has been tested with the latest client as well as clients with 2 versions back. It means that if changes are added to version 5.10, then the client on 5.10 as well as 5.9 and 5.8 must be tested!
- Please verify that PRs added to **core-subsetup** are following the requirements defined in Setup/delivery package requirements section above!

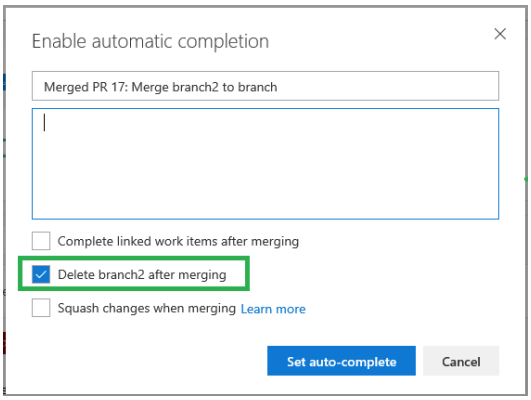
Guidelines for sending pull requests

When sending a pull request (PR) on one of the core- repositories; please observe the following:

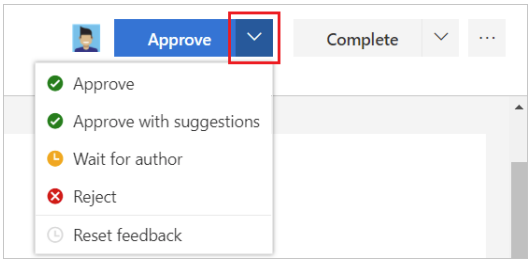
- Some of the core repositories (core-bl, core-bifstore) require two reviewers:
  - One of them has to be from the trusted reviewer group for the specific repo and will be added as a reviewer automatically.
  - The other reviewer should be someone else that knows, or should know about the part of the code base you have made changes to.
    - Add the appropriate team as a required reviewer for your PR. For example if you have made changes that affect eByggesak, add core-team-plan-and-build as a required reviewer.
    - It's expected that **all** developers contribute to increase the quality of the code that gets merged. If you have a question or a comment, don't hesitate to speak up.
    - If you don't understand something, add a comment on the PR and ask! It's important that the code we merge is as easy to understand as possible. Remember that you, or someone else, will be responsible for maintaining this code later.
- Write a description to the pull requests that provide the reviewers with helpful information. Examples:
  - Dependent/linked PRs from other repositories
  - In depth technical information about what you have done and why you have solved it like you did. For example: "I chose to use a byte array instead of a stream because.."
  - Other options you have considered
  - If some parts of the PR should be reviewed more carefully than other. For example if you are not sure that something is solved in the best possible way.
- Make sure that the linked task(s) contain a detailed functional description. This helps reviewers understand what you have done.
- If you are not used to git; **get familiar with how it works**. You can e.g. refer to: How to Git with Visual Studio and TFS
- **Always create a new git dev branch for each change.** Don't re-use dev branches for new changes. Delete each branch once it's merged.
- **Don't bundle multiple unrelated changes** into a single, huge PR. A PR should be for a single bugfix or functionality change. It should never bundle unrelated changes.
- **Do not push large updates to your PR after it has been approved**. We don't reset approvals upon pushing new code, so you (the developer) are responsible for not abusing this. Conversely, if you do a merge from master and get non-trivial merge conflicts, **please request a new review before merging!**
- **The code must be ready to ship.** In the future we will be releasing new versions from main/master branch every three weeks; and that means that all code committed to these branches must be in a state that is ready to ship at any time. That does not mean that the functionality needs to be ready; it could be in a disabled state using a feature switch. The important thing is that the code change does not cause any problems or side-effects.
- **Your branch should be up to date to avoid merge conflicts.** If you submit a PR on an outdated dev branch; it means you have a high chance of merge conflicts when merging into master. After the code review, the merge to master will be halted because of the conflict. You will then have to update your branch, get a *new* code review, before you can finally complete the PR. This wastes everyone's time. To avoid this type of

issue, please update your branch before submitting the PR. Refer to How to Git with Visual Studio and TFS for details.

- If you are making a change in a hotfix/release branch; you must **cherry-pick** from master. This means you must make the change on master first, *and wait for it to be merged*, before you can create the PR for the hotfix branch.
- When your PR is created; **look over the changes** in the files tab on the pull request page. Check that everything looks ok, and that you didn't add any unintended files or improperly merge changes. Additionally, before committing, please ensure you don't include anything you did not intend to. A good practice is to "stage" the files manually before committing to avoid unintentional changes.
- (If you have the permission) **set auto-completion with the 'delete branch' option.** This helps avoid a huge list of abandoned branches building up. If you don't have permission to set the 'delete branch' option however, you should not set auto-complete, nor should you manually complete a PR. (Let the reviewer do it).



- **If the PR shouldn't be merged right away; set the 'wait for author' option** on your PR. Usually, you should not send the PR at all if it's not ready to merge. However, there might be certain scenarios where you want to put the PR 'on hold' for a while. E.g. if you found you made a mistake, or if you need to wait for another change. The 'wait for author' option signals to the other reviewers that they need to wait; they shouldn't complete the PR right away. Clear the 'wait for author' option when you are ready.



- **Watch for updates** on your PR. Pull requests are not "fire and forget"; you'll need to follow what happens to it. Either by watching the DevOps notification emails, or by checking the PR status page manually. If rejected, or if any comments are added; you need to follow up appropriately by either revising your code, clarifying via comments, or by abandoning (deleting) the PR. If the build fails, it may be you need to correct errors, or add unit tests. If the PR got accepted, check that the PR actually was completed (there could be a merge issue). When the PR is successfully completed, you should follow up by deleting your local dev branch.

Build & Release Pipeline

The following must be considered while creating build & release pipeline

Yaml Basics

Before creating pipeline (build / release) definition, go thru basics of yaml. It is required when you design the pipeline.

1. yaml basics (https://docs.microsoft.com/en-us/azure/devops/pipelines/process/phases?view=azure-devops&tabs=yaml)

Build Pipeline

Build pipeline should be created only using yaml scripts, don't use traditional(UI based) build pipeline. Reference build pipeline(s)

1. core-bl (https://dev.azure.com/tieto-si/360/_apps/hub/ms.vss-build-web.ci-designer-hub?pipelineId=709&branch=master)
2. core-360 (https://dev.azure.com/tieto-si/360/_apps/hub/ms.vss-build-web.ci-designer-hub?pipelineId=736&branch=master)

Release Pipeline

If any release / deployment is associated with a build pipeline, use multi stage builds, don't use traditional(UI based) release pipeline. Reference multi stage / Release pipeline(s)

1. core-cloud-automation (https://dev.azure.com/tieto-si/360/_apps/hub/ms.vss-build-web.ci-designer-hub?pipelineId=797&branch=master)
2. Hello world (https://dev.azure.com/tieto-si/360/_apps/hub/ms.vss-build-web.ci-designer-hub?pipelineId=928&branch=master)

We will help you if required Team Automation (mailto:Automation-si@tieto.com)

Impact Analysis

The impact analysis will assure what part of an application need to be changed. The impact on the system is analyzed on Requirements, Design & Architecture, impact on Test, etc.

Best practices for change Impact Analysis

Continuous communication between developer and tester is must, not to miss any change needed to implement

Identify if any user interface changes, deletions or additions are required

Estimate the number of unit/integration test cases that will be required

Identify any impact of the proposed change to another modules/project

BL code updated/deleted (function name changed, parameters changes)

All references in BL repo as well as other repo like Core-BL, Core-BIFstore, React etc.. Should be checked

Dependent modules should be considered in testing of BL code

Changes Related to unit tests/integration tests should run and passed

API change – React repo should be checked for the dependency

Project specific change (but dependent on another project)

Ex – Progress plan folder is used within most of the projects, in case there is any change/modification in progress plan folder as per the requirement in specific project then that need to be shared/discussed with necessary teams/other teams who is also impacted or responsible for the same folder

Unit test/Integration Test and Auto test

Unit/integration tests should be written for the new code or modified code

Auto test should be written for competed features.

We have daily pipeline running for all tests, in case there is any discrepancy then tests will fail and this will be back tracked to find the root cause why its failing

DB related change and impact

Whenever there is any change related to DB objects like Stored Procedures, functions, tables, views etc.. Need to make sure impact has been checked on dependent entities/objects from DB as well as core bl and other repo's

Hotfix – Hotfix routine need to be followed for Hotfix and its impact

Check for backward combability and .Net versions. Latest is in .Net 6, so some libraries might not be available in older version of .Net

We need to make sure we are raising correct hotfix PR as well as adding correct files Ex.. DLL for correct hotfix version for CI build pipeline

SSU Sub - Setup Tool

SSU tool is connected with master DB with concurrent access

SSU tool not generating only required changes but more of the changes, which is not done that can also be shown in changed files

Multiple members can work at same time on SSU tool so changes can be available in multiple areas of PR

This need to be checked in detail and need to make sure only the change that we did that should be part of PR

In case found any other change not related to the expected change then need to highlight this to respective teams/members before raising PR

Performance

Impact need to be checked for performance, new change its either new functionality or changing existing functionality or a bug fix should not impact on performance

This is especially important in new React-views that populate data client-side with new Client API calls.

For more details, check Coding for Performance

Good To Have

Dependency sheet can be prepared for changed module and impacted modules. This will help in testing from developer side as well as QA side

Impact analysis should be attached or added in comment section on US/Bug. It will be easy for QA team to know what should be tested along with impacted parts

---